

CI/CD Tooling & Practices - Part 2

WELCOME TO CLASS 8!



Goals of the day



Review sample Python application



Create CI pipeline for this app



Use Ansible for code deployment

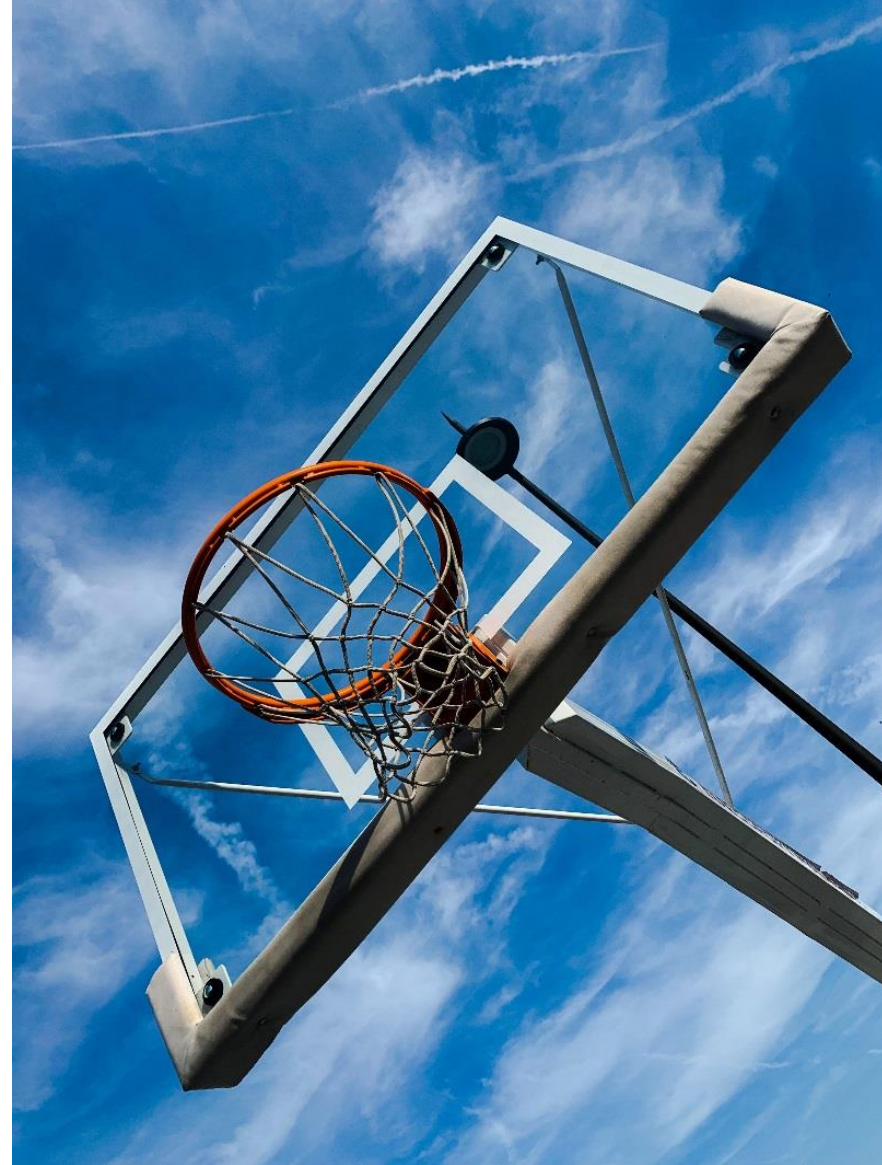


Photo by [David Becker](#) on [Unsplash](#)

Agenda

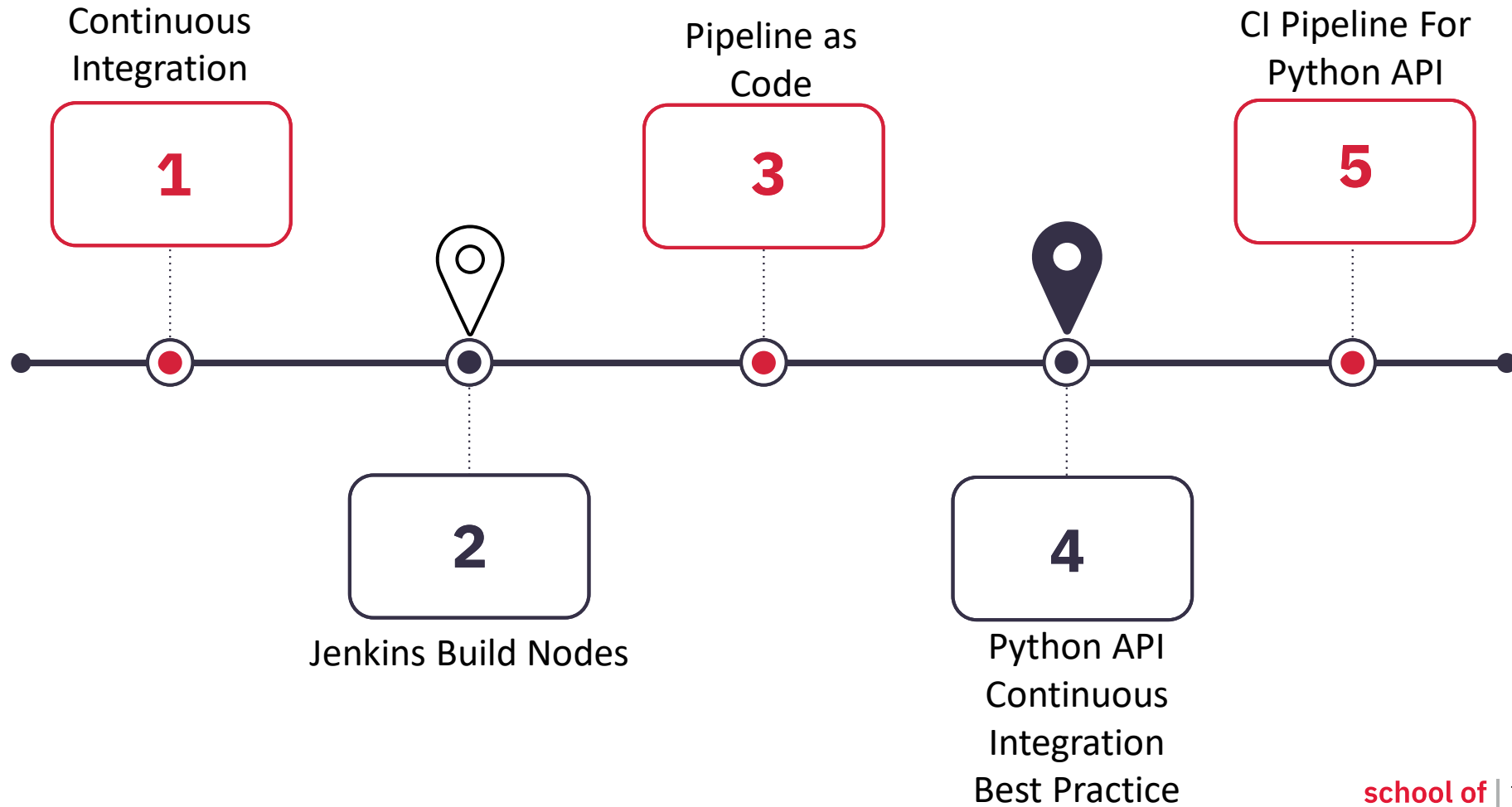
- In depth Continuous Integration
- Jenkins Server with Dedicated Build Nodes
- **Break**
- Managing multiple servers with Ansible
- Using Ansible to deploy code
- Ansible roles
- Lab overview

Photo by David Becker on Unsplash

Review

- How did the Jenkins install from last class go for you?
- What else do you recall from the last class about software development?
- As a CloudOps engineer, which practices would you like to see from developers of applications running on your platform?
- Brief Q & A

Journey Map



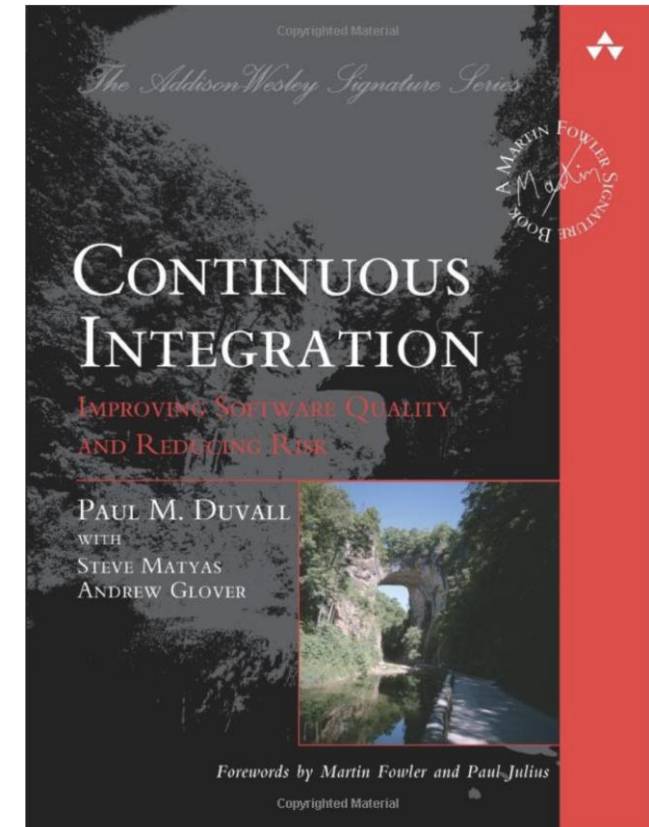
In Depth Continuous Integration

Continuous Integration [from last week]

To realise Fowler's vision, you need:

- Source code management system
- Agreed branching strategy
- Build script executable from command line
- Automated testing
- Clear approach for managing database changes
- Team culture on broken builds
- Regular commits from all team members
- Continuous improvement mindset

Let's review each item in more detail.



Continuous Integration

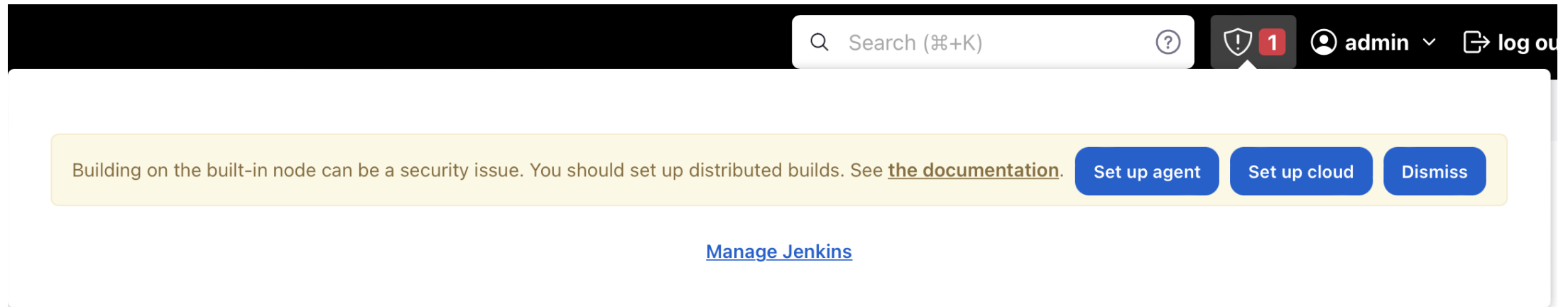
- In the previous class, when you created your Jenkins server, you may have noticed a security warning related to running Jenkins workloads on the master server.
- Best practice is to create Jenkins nodes or agents, on which workloads are run, which are controlled by the Jenkins master.
- In today's class, we will walk through how to create and configure dedicated Python and Java nodes and execute build jobs on each.

Continuous Integration (con't)

- In the previous class, when you created your Jenkins server, you may have noticed a security warning related to running Jenkins workloads on the master server.
- Best practice is to create Jenkins nodes or agents, on which workloads are run, which are controlled by the Jenkins master.
- In today's class, we will walk through how to create and configure dedicated Python and Java nodes and execute build jobs on each.

Next, Jenkins Server with Dedicated Build Nodes

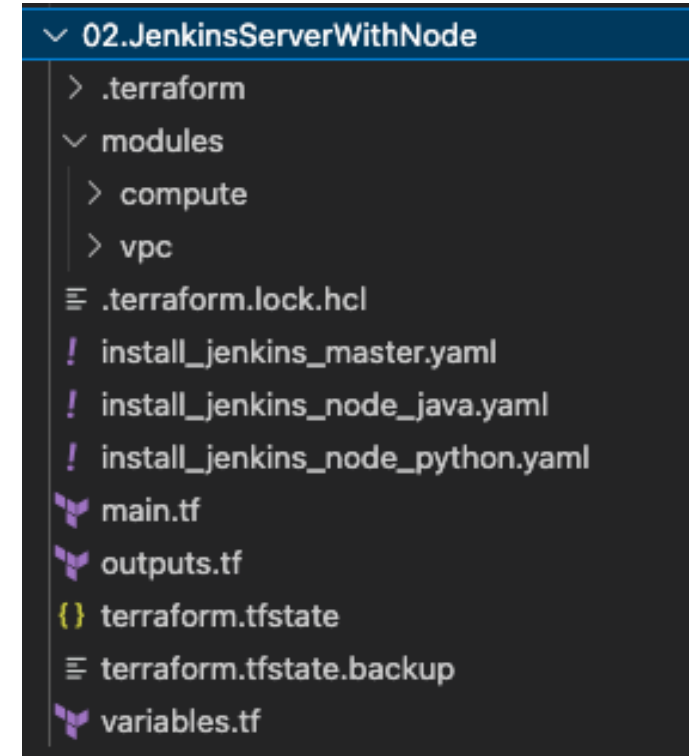
Jenkins Server with Dedicated Build Nodes 1 of 8)



- In the previous class, when you created your Jenkins server, you may have noticed a security warning related to running Jenkins workloads on the master server.
- Best practice is to create Jenkins nodes or agents, on which workloads are run, which are controlled by the Jenkins master.
- In today's class, we will walk through how to create and configure dedicated Python and Java nodes and execute build jobs on each.

Jenkins Server with Dedicated Build Nodes (2 of 8)

- Execute terraform (and underlying Ansible code) to create 3 EC2s as follows:
 - Jenkins Master
 - Jenkins Java Node
 - Jenkins Python Node
- While provisioning and configuration is progressing, review code to understand what is created and how it is configured.
- Check the output for IPs for the nodes and URL for the master server. Complete configuration of server as per last class.



Jenkins Server with Dedicated Build Nodes (3 of 8)

- Check that both agents have been setup correctly by connecting over SSH and running the following commands

- Jenkins Java Node

- mvn -version
- java -version

```
[ec2-user@ip-10-0-1-116 ~]$ java -version
openjdk version "11.0.20" 2023-07-18 LTS
OpenJDK Runtime Environment (Red_Hat-11.0.20.0.8-1.amzn2.0.1) (build 11.0.20+8-LTS)
OpenJDK 64-Bit Server VM (Red_Hat-11.0.20.0.8-1.amzn2.0.1) (build 11.0.20+8-LTS, mixed mode,
[ sharing)
[ec2-user@ip-10-0-1-116 ~]$ java -version
openjdk version "11.0.20" 2023-07-18 LTS
OpenJDK Runtime Environment (Red_Hat-11.0.20.0.8-1.amzn2.0.1) (build 11.0.20+8-LTS)
OpenJDK 64-Bit Server VM (Red_Hat-11.0.20.0.8-1.amzn2.0.1) (build 11.0.20+8-LTS, mixed mode,
[ sharing)
[ec2-user@ip-10-0-1-116 ~]$ mvn -version
Apache Maven 3.9.4 (dfbb324ad4a7c8fb0bf182e6d91b0ae20e3d2dd9)
Maven home: /opt/apache-maven-3.9.4
Java version: 11.0.20, vendor: Red Hat, Inc., runtime: /usr/lib/jvm/java-11-openjdk-11.0.20.
0.8-1.amzn2.0.1.x86_64
Default locale: en_CA, platform encoding: UTF-8
OS name: "linux", version: "4.14.322-244.539.amzn2.x86_64", arch: "amd64", family: "unix"
[ec2-user@ip-10-0-1-116 ~]$
```

- Jenkins Python Node

- python3 --version
- pip3 -V

```
[ec2-user@ip-10-0-1-172 ~]$ python3 --version
Python 3.7.16
[ec2-user@ip-10-0-1-172 ~]$ pip3 -V
pip 20.2.2 from /usr/lib/python3.7/site-packages/pip (python 3.7)
[ec2-user@ip-10-0-1-172 ~]$
```


Jenkins Server with Dedicated Build Nodes (4 of 8)

- Next step is to configure SSH connection between each Jenkins node and the Jenkins master.
- Start by SSH to the Jenkins master and complete the

following:

- a) `sudo su -s /bin/bash jenkins`
- b) `mkdir /var/lib/jenkins/.ssh`
- c) copy & execute both "`ssh-keyscan - ...known_hosts`" commands from terraform output
- d) `cd /var/lib/jenkins/.ssh`
- e) `chown jenkins:jenkins known_hosts && chmod 700 known_hosts`

```
[bash-5.2$ ssh-keyscan -H 10.0.1.53 >>/var/lib/jenkins/.ssh/known_hosts
# 10.0.1.53:22 SSH-2.0-OpenSSH_8.7
# 10.0.1.53:22 SSH-2.0-OpenSSH_8.7
# 10.0.1.53:22 SSH-2.0-OpenSSH_8.7
# 10.0.1.53:22 SSH-2.0-OpenSSH_8.7
# 10.0.1.53:22 SSH-2.0-OpenSSH_8.7
```

Jenkins Server with Dedicated Build Nodes (5 of 8)

- Then configure each Jenkins node. We will cover Python node, repeat for Java node.
- SSH to the Jenkins node and complete the following:
 - a) `sudo su - python-node`
 - b) `ssh-keygen -t ed25519 -N "" -f /home/python-node/.ssh/id_ed25519`
 - c) `cd .ssh && cat id_ed25519.pub > authorized_keys`
 - d) `chmod 700 authorized_keys`
 - e) `cat id_ed25519` <you will need this key later to setup the user which the Jenkins master uses to connects to this node>

```
[ec2-user@ip-10-0-1-53 ~]$ sudo su - python-node
[python-node@ip-10-0-1-53 ~]$ ssh-keygen -t ed25519 -N "" -f /home/python-node/.ssh/id_ed25519
Generating public/private ed25519 key pair.
Created directory '/home/python-node/.ssh'.
Your identification has been saved in /home/python-node/.ssh/id_ed25519
Your public key has been saved in /home/python-node/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:sxBMVhOJIskoCDjtYYbJiCEpSKm/8jSwzSRuxvvhp4 python-node@ip-10-0-1-53.ec2.internal
The key's randomart image is:
+--[ED25519 256]--+
|%00...+B.         |
|0=o.o0            |
|**+. +           |
|B *               |
|.. o . S          |
|. + . o           |
| +O+             |
|..**             |
| oE=+            |
+----[SHA256]-----+
[python-node@ip-10-0-1-53 ~]$ cd .ssh && cat id_ed25519.pub > authorized_keys
[python-node@ip-10-0-1-53 .ssh]$ chmod 700 authorized_keys
```

Jenkins Server with Dedicated Build Nodes (6 of 8)

- Now we can move to the Jenkins UI. Click the 'Setup Agent' button which is displayed when you click on the security warning.
- Enter Python-Node as name, with remaining details as per this screenshot. Note you will need to create the Credentials as a new user.
 - Click the Add button, select Jenkins and then configure a user of kind "SSH Username with private key". ID = python-node and enter the private key directly from the output from the `cat id_ed25519` command.

Number of executors ?

1

Remote root directory ?

/home/python-node/Jenkins

Labels ?

Python-Node

Usage ?

Use this node as much as possible

Launch method ?

Launch agents via SSH

Host ?

10.0.1.172

Credentials ?

python-node

Add

Host Key Verification Strategy ?

Known hosts file Verification Strategy

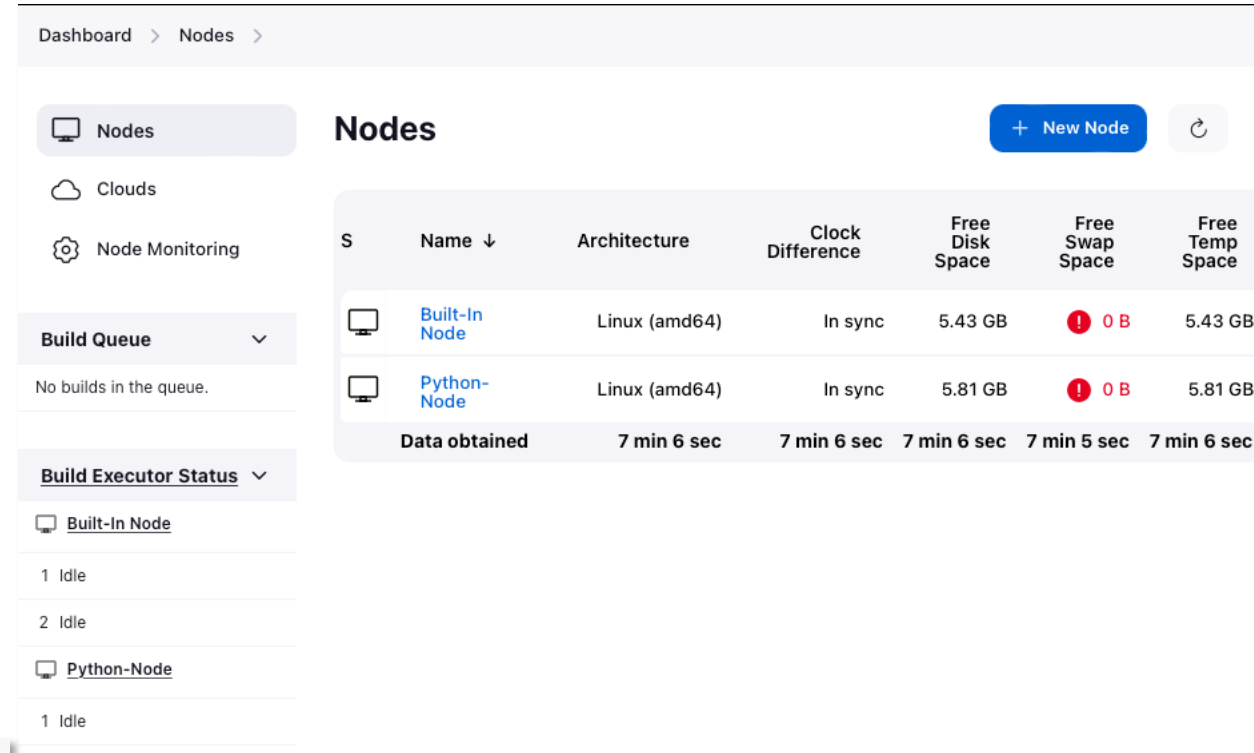
Advanced Edited

Availability ?

Keep this agent online as much as possible

Jenkins Server with Dedicated Build Nodes (7 of 8)

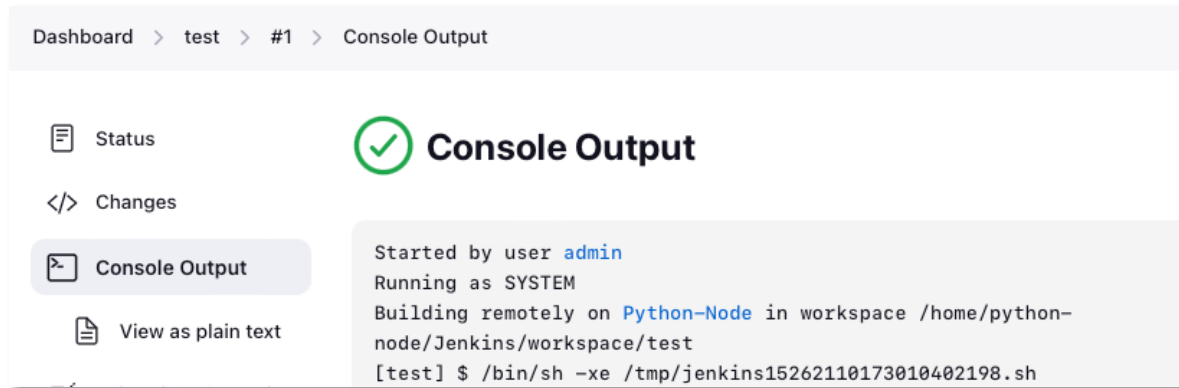
- Once configured, click on the Build Executor Status link on left hand side of the home page to show available nodes.
- Click Built-In Node, then configure. Set executors to zero, then return to Build Executor Status page to see just the Python-Node available.
- Create a simple freestyle job and execute. In the console out logs for the first invocation of the job, you will see it was run on our new node.



The screenshot shows the Jenkins 'Nodes' page. On the left sidebar, there are links for 'Nodes', 'Clouds', and 'Node Monitoring'. Below these are sections for 'Build Queue' (showing 'No builds in the queue.') and 'Build Executor Status'. The main area is titled 'Nodes' and features a '+ New Node' button. It contains a table with the following data:

S	Name ↓	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space
	Built-In Node	Linux (amd64)	In sync	5.43 GB	0 B	5.43 GB
	Python-Node	Linux (amd64)	In sync	5.81 GB	0 B	5.81 GB
Data obtained		7 min 6 sec	7 min 6 sec	7 min 6 sec	7 min 5 sec	7 min 6 sec

Below the table, there are sections for 'Built-In Node' (showing 1 Idle executor) and 'Python-Node' (showing 1 Idle executor).



The screenshot shows the Jenkins 'Console Output' page for a job named 'test'. The page has a breadcrumb trail: 'Dashboard > test > #1 > Console Output'. On the left sidebar, there are links for 'Status', 'Changes', and 'Console Output' (which is selected). Below these is a 'View as plain text' link. The main area shows a green checkmark icon and the title 'Console Output'. The output text is as follows:

```
Started by user admin
Running as SYSTEM
Building remotely on Python-Node in workspace /home/python-node/Jenkins/workspace/test
[test] $ /bin/sh -xe /tmp/jenkins15262110173010402198.sh
```

Jenkins Server with Dedicated Build Nodes (8 of 8)

- When you have completed the Python-Node, then repeat for the Java-Node and set the executors to zero for the Built-In Node.
- This results in just 2 nodes available.
- From here we will configure a build job for a simple Spring Boot application to run on the Java-Node.
- Later in the class we will also use the Python node.

Build Executor Status ▾	
 <u>Java-Node</u>	
1 Idle	
 <u>Python-Node</u>	
1 Idle	

Sample build pipeline for Spring Boot REST API

Spring Boot REST API (2 of 9)

- For this next section, we will build a simple Spring Boot REST API using a Jenkins pipeline project.
- Before getting into the Jenkins part, let's first look at the application code.
- As you can see, its very straightforward REST API, the supports two GET calls and two POST calls.
- We will validate these calls at the end of this section by running this code.
- On the next slide, we will now review the JenkinsFile, which is in the root folder of this repo and captures the Jenkins build activities for this application.

```
package com.example.springboot;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/")
    public String index() {
        return "Greetings from Spring Boot!";
    }

    @GetMapping("/DevOps")
    public String DevOps() {
        return "Flow; fast feedback; learning & experimentation";
    }

    @PostMapping("/DevOps")
    public String DevOps(@RequestParam(value = "name", defaultValue = "Unit") String name) {
        return name + " testing";
    }

    @PostMapping("/YorkU")
    public String YorkU(@RequestParam(value = "name", defaultValue = "Zoom") String name) {
        return name + " Day";
    }
}
```


Spring Boot REST API (1 of 9)

- Fork the Spring Boot app to your Git Hub account.
- Create a Git Hub access token, which we will use to configure pipeline access to this forked repo.
 - Click your profile icon and select Settings from this menu
 - Click Developer Settings at the end of menu on left hand side of next screen
 - Click *Personal access tokens > Tokens (classic) > Generate new token (classic)*
 - Add Jenkins in Note field and select *admin:repo_hook, repo*
 - Click Generate token and record the result..



This Photo by Unknown Author is licensed under CC BY

Spring Boot REST API (3 of 9)

- The structure of the JenkinsFile is as follows*.

agent specifies the node on which the job will run

parameters choice enable input to be gathered before the job is run, which can control the execution flow of the job later

stages are the main part of the pipeline. In this case the run various shell commands to build the code, report on the folder content post build and copy the built artifact to another folder

post enables steps to be configured for after completion of the main body of the job

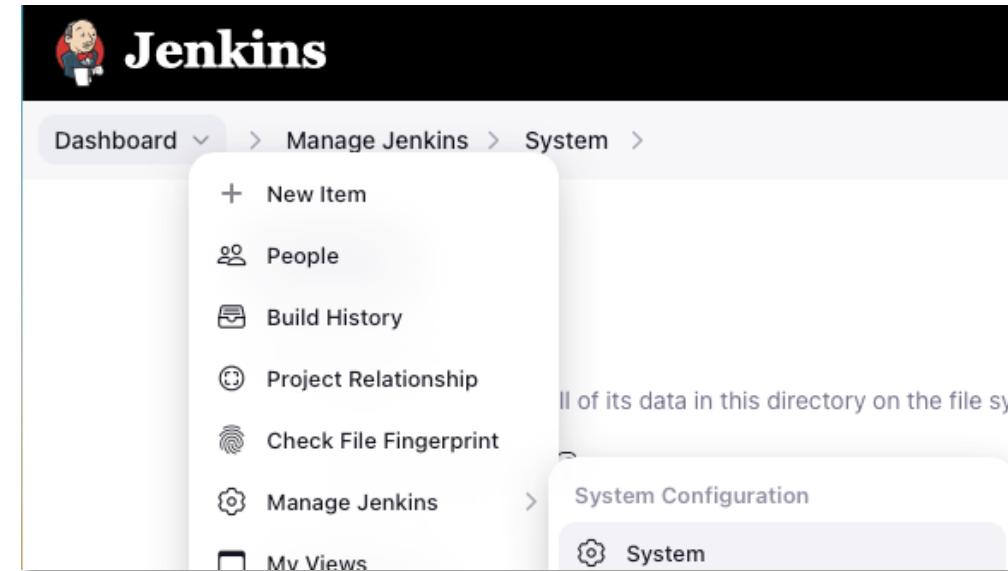
```
1 pipeline {
2   agent {
3     node {
4       label 'Java-Node'
5     }
6   }
7   parameters {
8     choice(
9       choices: ['true' , 'false'],
10      description: 'Do you wish to save the jar after the build?',
11      name: 'SAVE_JAR')
12   }
13   stages {
14     stage('build') {
15       steps {
16         sh 'mvn clean install'
17       }
18     }
19     stage('check workspace details') {
20       steps {
21         sh 'pwd && ls -lrt'
22       }
23     }
24     stage('copy latest jar'){
25       when {
26         expression { params.SAVE_JAR == 'true' }
27       }
28       steps {
29         sh 'mkdir -p ~/jars && cp ./target/*.jar ~/jars'
30       }
31     }
32   }
33   post {
34     success {
35       echo 'Job completed successfully'
36     }
37     failure {
38       echo 'Job failed'
39     }
40   }
41 }
```

* <https://www.jenkins.io/doc/book/pipeline/jenkinsfile/>

Spring Boot REST API

(4 of 9)

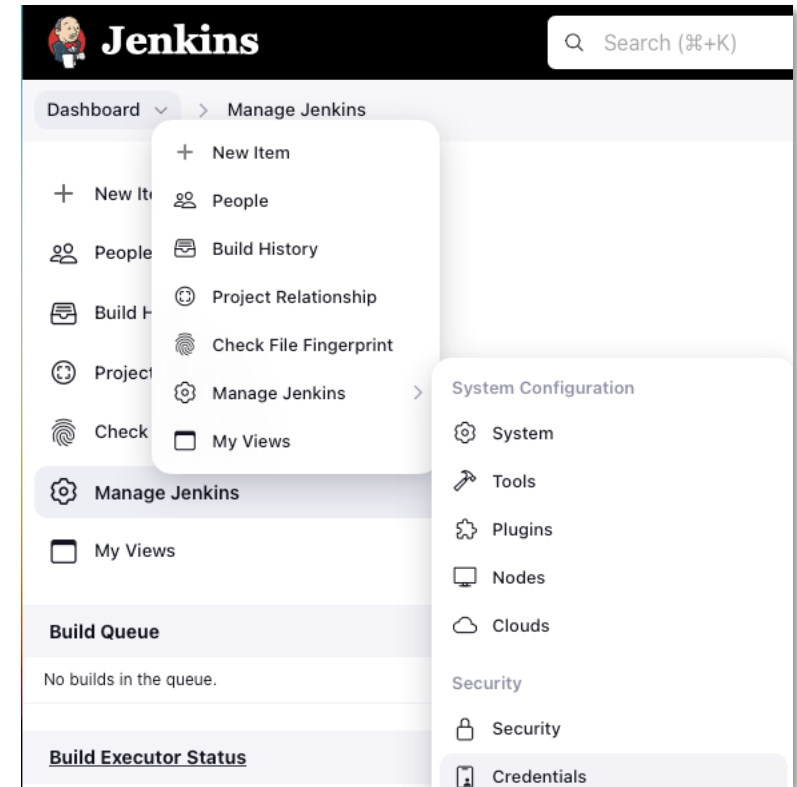
- Now we return to the Jenkins setup for this application.
- You need to configure an access key in Jenkins so that builds can access Git Hub repos. Click on System through the menus shown in this screenshot.
- On the settings page, scroll down to find the GitHub option and add a server.
- Add credentials, entering the token as a type secret text, with credentials called GitHubKey. Test the connection.
- Click Save at the end of the page.

A screenshot of the 'GitHub Servers' configuration page in Jenkins. The page has a title 'GitHub Servers' with a help icon. Below it is a section 'GitHub Server' with a red close button. The form contains the following fields: 'Name' (with a help icon) containing 'Github'; 'API URL' (with a help icon) containing 'https://api.github.com'; 'Credentials' (with a help icon) containing 'GitHubKey' and a dropdown arrow; an 'Add' button; a status message 'Credentials verified for user ShaneMannion, rate limit: 4999'; a 'Test connection' button; a checkbox for 'Manage hooks'; and an 'Advanced' dropdown menu.

Spring Boot REST API

(5 of 9)

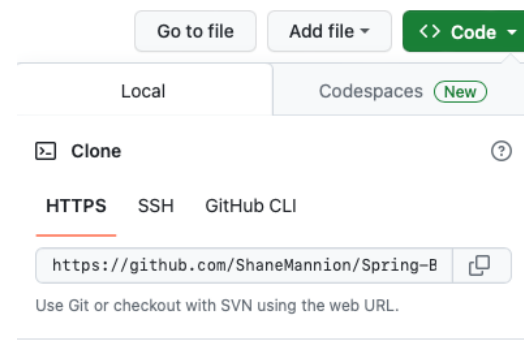
- Next add credentials that are linked to the GitHub token from previous slide. Select menu item as per the screenshot.
- Click through on the global hyperlink under Domain, then 'Add Credentials', with username same as your GitHub username and password same as the personal access token generated in GitHub. Set ID as GitHubID.
- Create a new job, of type Pipeline, with name 'Spring Boot Build'.



Spring Boot REST API

(6 of 9)

- Scroll to pipeline section and select 'Pipeline script from SCM' from the Definition drop down.
- Select 'Git' as the SCM, copy the HTTPS URL from your forked repo in GitHub.
- Select credentials added in previous slide and specify the branch that contains the code you want to build.
- Click Save and Build Now on the next screen



Pipeline

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/ShaneMannion/Spring-Boot-Sample.git

Credentials ?

shanemannion/*****

Add ▾

Advanced ▾

Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/notUsingLibrary

Add Branch

Repository browser ?

(Auto)

Additional Behaviours

Add ▾

Script Path ?

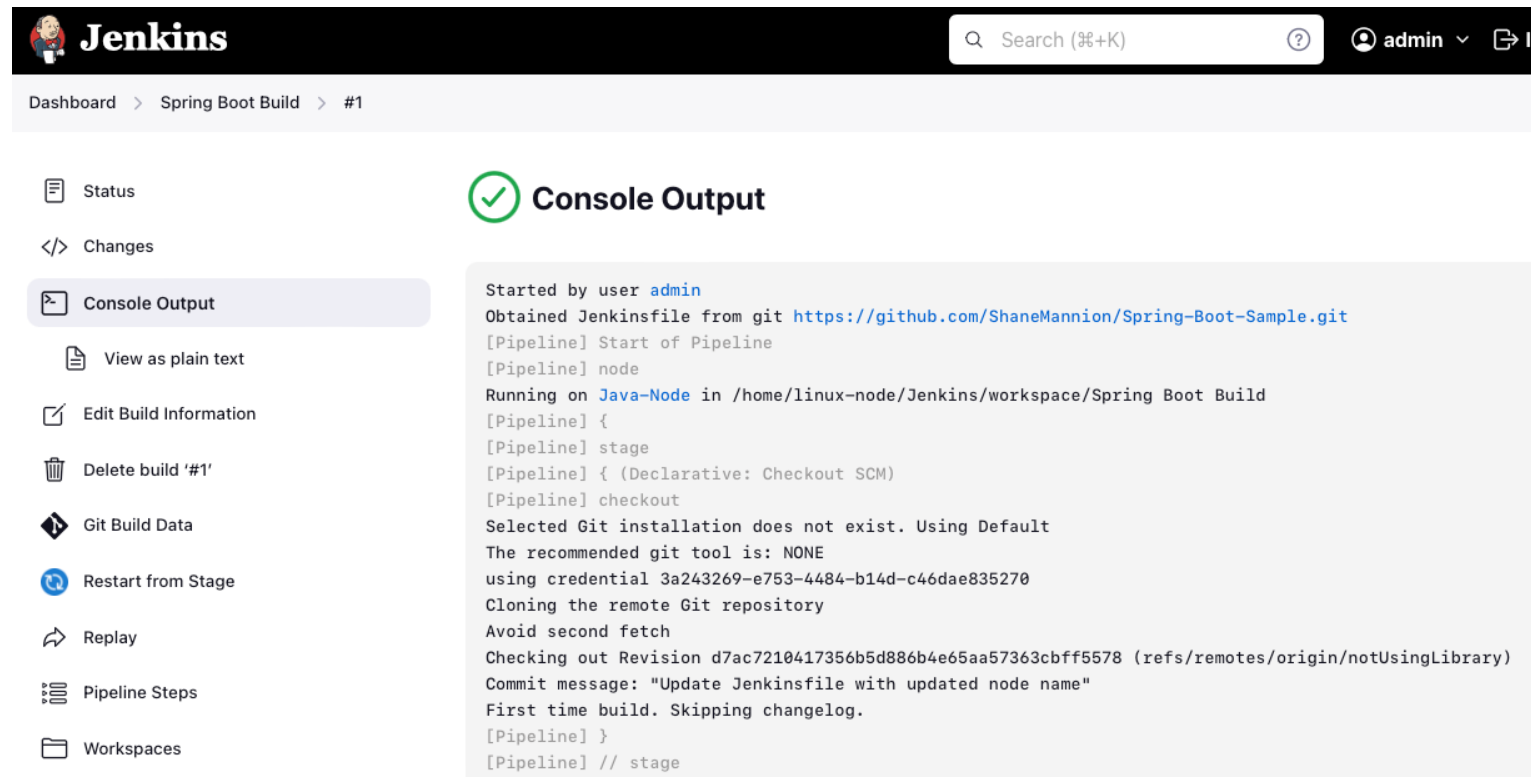
Jenkinsfile

☒ Lightweight checkout ?

Save Apply

Spring Boot REST API (7 of 9)

- Once this first execution has completed, click on job #1 and review the console output.
- Note that it ran on Java-Node. The first execution will be very verbose, with a lot of download activity from maven captured.



The screenshot shows the Jenkins web interface. At the top is the Jenkins logo and a search bar. Below the header, the breadcrumb trail reads 'Dashboard > Spring Boot Build > #1'. On the left sidebar, the 'Console Output' option is selected. The main area displays the console output for the build, which is titled 'Console Output' with a green checkmark icon. The output text is as follows:

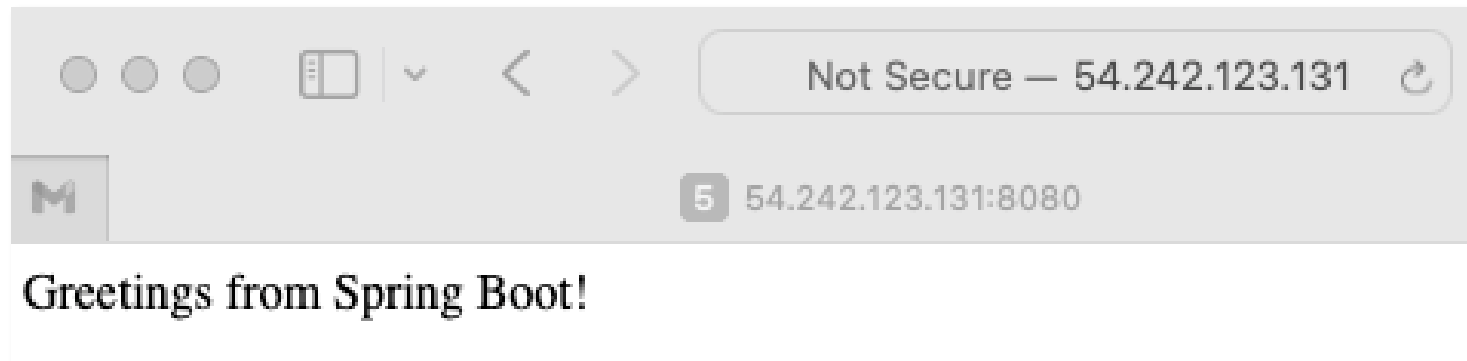
```
Started by user admin
Obtained Jenkinsfile from git https://github.com/ShaneMannion/Spring-Boot-Sample.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Java-Node in /home/linux-node/Jenkins/workspace/Spring Boot Build
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
using credential 3a243269-e753-4484-b14d-c46dae835270
Cloning the remote Git repository
Avoid second fetch
Checking out Revision d7ac7210417356b5d886b4e65aa57363cbff5578 (refs/remotes/origin/notUsingLibrary)
Commit message: "Update Jenkinsfile with updated node name"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
```

Spring Boot REST API (8 of 9)

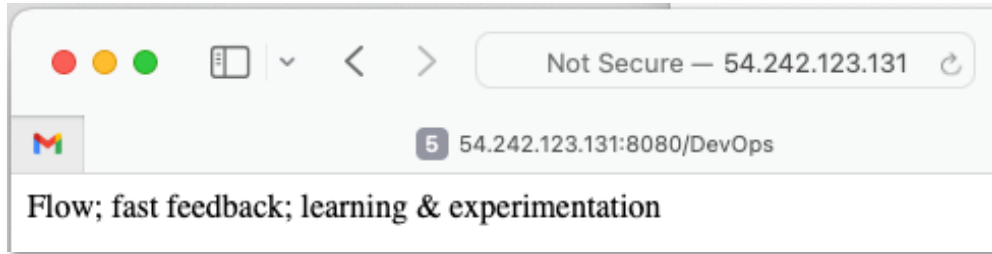
- Run the job a second time. Review the console output for this execution and note how the stages of the output map to the different parts of the Jenkinsfile. This console output is where you go to troubleshoot any issues in running the job. Here the output provides details of build success, folder contents and copy operation for the jar.
- Navigate to the jar copy destination folder on the Java-Node, run the following command

```
java -jar spring-boot-initial-0.0.1-SNAPSHOT.jar
```

- Load the Public IP of the node with port 8080 into your browser to see the content in the screenshot below.



Spring Boot REST API (9 of 9)



- The POST methods can be invoked from the command line as follows.

```
shane@Shanes-MacBook-Pro code % curl -X POST http://54.242.123.131:8080/DevOps -d "name=value"  
[value testing]  
shane@Shanes-MacBook-Pro code % curl -X POST http://54.242.123.131:8080/YorkU -d "name=School"  
[School Day]  
shane@Shanes-MacBook-Pro code % curl -X POST http://54.242.123.131:8080/DevOps  
Unit testing
```

```
package com.example.springboot;
```

```
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.PostMapping;  
import org.springframework.web.bind.annotation.RequestParam;  
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
```

```
public class HelloController {
```

```
    @GetMapping("/")
```

```
    public String index() {
```

```
        return "Greetings from Spring Boot!";
```

```
    }
```

```
    @GetMapping("/DevOps")
```

```
    public String DevOps() {
```

```
        return "Flow; fast feedback; learning & experimentation";
```

```
    }
```

```
    @PostMapping("/DevOps")
```

```
    public String DevOps(@RequestParam(value = "name", defaultValue = "Unit") String name) {
```

```
        return name + " testing";
```

```
    }
```

```
    @PostMapping("/YorkU")
```

```
    public String YorkU(@RequestParam(value = "name", defaultValue = "Zoom") String name) {
```

```
        return name + " Day";
```

```
    }
```

```
}
```


Sample Python Application to Demonstrate Continuous Integration Best Practice

Sample Python Application (1 of 11)

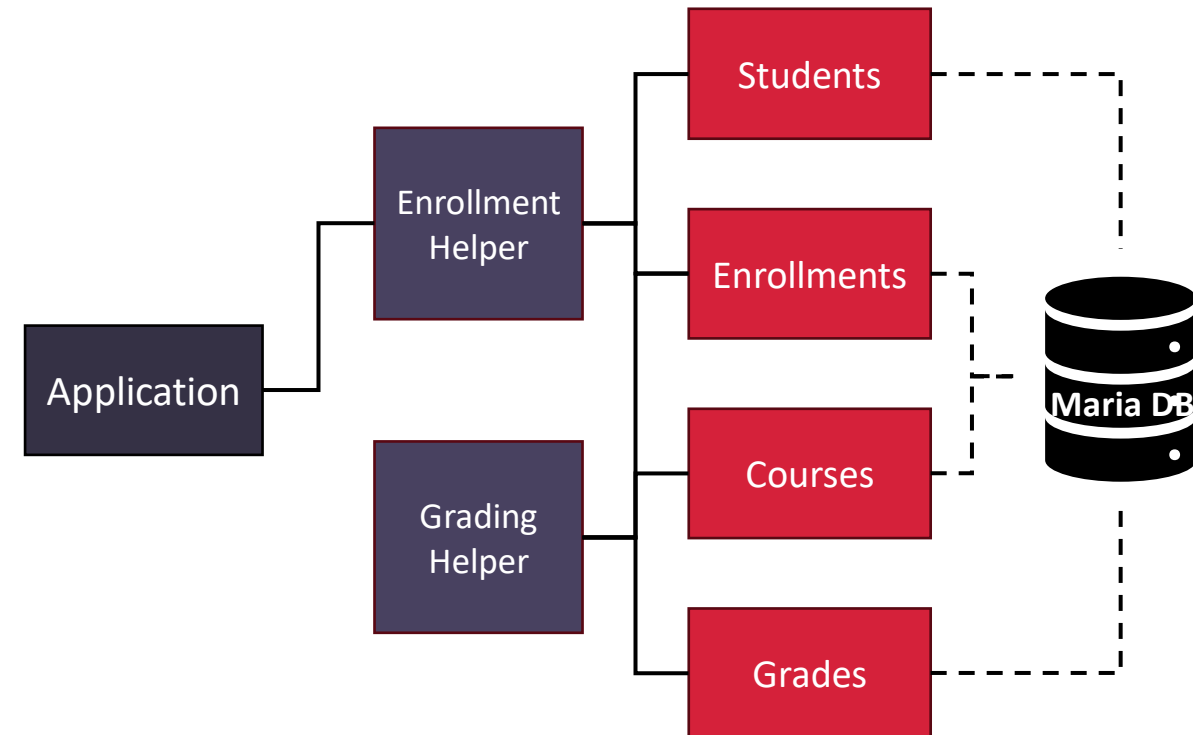
- The previous Spring Boot Sample was very simple. To better understand CICD practices, it is helpful to have a more complex application to work with.
- Understanding applications and how they are structured and tested will help you as a CloudOps engineer in hosting applications effectively e.g., assessing which applications are suitable for blue-green deployments.
- We will review an incomplete application written in Python to understand characteristics that are important from a DevOps/CloudOps perspective.



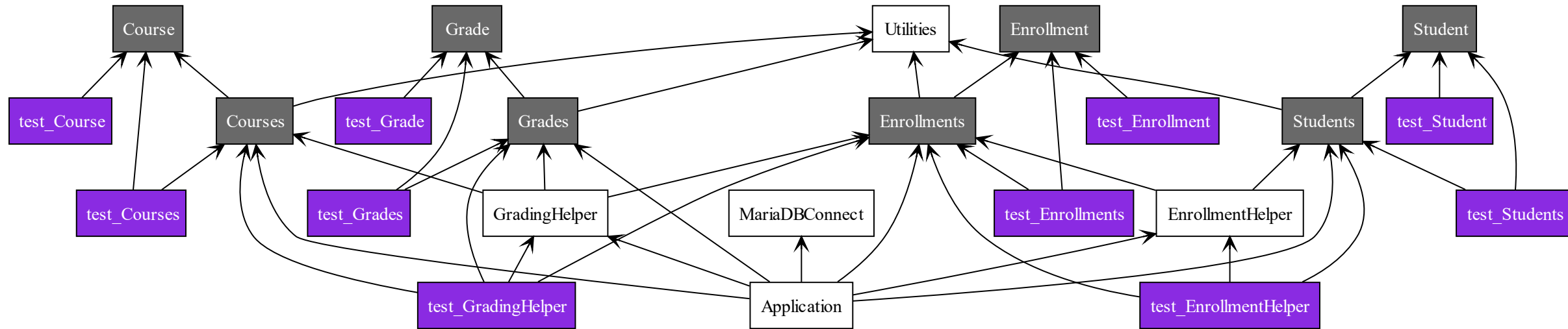
This Photo by Unknown Author is licensed under CC BY

Sample Python Application (2 of 11)

- Our sample application is used for student enrollments and grades.
- It uses Maria DB to persist data and is based on an architecture which is decomposed into layers and distinct classes that each fulfill specific functions.
- The red boxes map directly to tables in the database.
- Logic to combine data across tables is codified in the Helper classes.
- The application class provides REST API access to the Helper classes.



Sample Python Application (3 of 11)



Legend

Unit Test

Data Layer

- This is a more detailed view of which class dependencies.
- You can see clearly that there are unit test classes for almost all the application code.
- 80% is generally accepted as optimal for unit test coverage. *
- Check instructions at start of checked in packages.dot file for details of how to generate this class diagram.

Sample Python Application (4 of 11)

```
shane@Shanes-MacBook-Pro Python % pytest -v
===== test session starts =====
platform darwin -- Python 3.11.5, pytest-7.2.0, pluggy-1.0.0 -- /opt/homebrew/opt/python@3.11/bin/python3.11
cachedir: .pytest_cache
rootdir: /Users/shane/Documents/Shane/yorkU/courses/CloudOpsCertificate/Course4/ClassMaterials/Cla
ss789/code/StudentGradesApp/Python
collected 15 items

test_Course.py::testCreateCourseAndReturnName PASSED [ 6%]
test_Courses.py::testBulkLoadCourses PASSED [ 13%]
test_Courses.py::testAddOneCourse PASSED [ 20%]
test_Enrollment.py::testCreateEnrollmentAndReturnDetails PASSED [ 26%]
test_EnrollmentHelper.py::testListStudentsByCourseID PASSED [ 33%]
test_Enrollments.py::testBulkLoadEnrollments PASSED [ 40%]
test_Enrollments.py::testAddOneEnrollment PASSED [ 46%]
test_Grade.py::testCreateGradeAndReturnDetails PASSED [ 53%]
test_Grades.py::testBulkLoadGrades PASSED [ 60%]
test_Grades.py::testAddOneGrade PASSED [ 66%]
test_GradingHelper.py::testListStudentsByCourseID_OneGrade PASSED [ 73%]
test_GradingHelper.py::testListStudentsByCourseID_ManyGrades PASSED [ 80%]
test_Student.py::testCreateStudentAndReturnSurname PASSED [ 86%]
test_Students.py::testBulkLoadStudents PASSED [ 93%]
test_Students.py::testAddOneStudent PASSED [100%]

===== 15 passed in 0.02s =====
```

Unit tests are easy to run and should be very fast. They are the most efficient way to regression test an application, with manual testing being the least.

The pytest utility selects source files that start with test*.py or *test.py, executing functions in these source files that start with test*.

Sample Python Application (5 of 11)

```
test_Enrollments.py ×
StudentGradesApp > Python > test_Enrollments.py > ...
1 import csv
2 from Enrollment import Enrollment
3 from Enrollments import Enrollments
4 import pytest
5
6 enrollments = Enrollments()
7
8
9 def testBulkLoadEnrollments():
10     enrollments.allEnrollments.clear()
11     assert enrollments.bulkLoad() == 10
12
13 def testAddOneEnrollment():
14     enrollment = Enrollment(1, 2, 3)
15     assert enrollments.addEnrollment(enrollment) == True
16
17
```

This is a simple test class, which tests both bulk load and single enrollment functions. Based on a check that the code execution is part of a PyTest run, the application code reads data from a csv file for bulk load,.

```
Enrollments.py •
StudentGradesApp > Python > Enrollments.py > Enrollments > bulkLoad
5 class Enrollments:
6     ...
7     ...allEnrollments = []
8
9     ...def addEnrollment(self, enrollment):
10     ...     if enrollment.enrollment_id > 0:
11     ...         self.allEnrollments.append(enrollment)
12     ...         return True
13     ...     else:
14     ...         return False
15
16
17     ...def bulkLoad(self, conn = None):
18     ...     rowCounter = 0
19     ...     if Utilities.isItTest():
20     ...         with open('../TestData/SampleEnrollmentData.csv', 'r') as csvfile:
21     ...             readCSV = csv.reader(csvfile, delimiter=',')
22     ...             try:
23     ...                 for row in readCSV:
24     ...                     rowCounter = rowCounter + 1
25     ...                     if rowCounter > 1:
26     ...                         enrollment_id= row[0]
27     ...                         course_id= row[1]
28     ...                         student_id=row[2]
29     ...                         enrollment = Enrollment(enrollment_id, course_id, student_id)
30     ...                         self.allEnrollments.append(enrollment)
31     ...                         lastRow = row
32     ...             except:
33     ...                 print("RowCounter: !> 1")
34     ...                 print("%s" % format(lastRow))
35
```

Testing code this way allows you to create well designed classes that are then added into a more complex app.

Building a regression unit test pack is also a huge benefit.

Sample Python Application (6 of 11)

The test data here is stored in a csv file and is also checked into version control along with the application code.

With this approach you can create as much test data as you wish, including edge scenarios, which increases confidence.

Another advantage of this approach is that the test data is fixed and therefore reliable. Test failures due to aged test data are expensive, as these failures are usually treated as bugs until the true root cause is identified.

StudentGradesApp > TestData > SampleEnrollmentData.csv

	Enrollment_ID,Course_ID,Student_ID
1	
2	1,1,1
3	2,2,1
4	3,3,1
5	4,4,1
6	5,1,2
7	6,2,2

```
Enrollments.py
StudentGradesApp > Python > Enrollments.py > Enrollments > bulkLoad
5 class Enrollments:
6     ...
7     ... allEnrollments = []
8
9     ... def addEnrollment(self, enrollment):
10         ... if enrollment.enrollment_id > 0:
11             ... self.allEnrollments.append(enrollment)
12             ... return True
13         ... else:
14             ... return False
15
16     ...
17     ... def bulkLoad(self, conn = None):
18         ... rowCounter = 0
19         ... if Utilities.isItTest():
20             ... with open('../TestData/SampleEnrollmentData.csv', 'r') as csvfile:
21                 ... readCSV = csv.reader(csvfile, delimiter=',')
22                 ... try:
23                     ... for row in readCSV:
24                         ... rowCounter = rowCounter + 1
25                         ... if rowCounter > 1:
26                             ... enrollment_id= row[0]
27                             ... course_id= row[1]
28                             ... student_id=row[2]
29                             ... enrollment = Enrollment(enrollment_id, course_id, student_id)
30                             ... self.allEnrollments.append(enrollment)
31                             ... lastRow = row
32                 ... except:
33                     ... print("RowCounter != 1")
34             ... print("%d" % format(lastRow))
35
```

Sample Python Application (7 of 11)

```
StudentGradesApp > Python > test_GradingHelper.py > ...
3 from Enrollments import Enrollments
4 from GradingHelper import GradingHelper
5
6 grades = Grades()
7 courses = Courses()
8 enrollments = Enrollments()
9 gradingHelper = GradingHelper(grades, courses, enrollments)
10
11
12 def loadTestData():
13     grades.allGrades.clear()
14     courses.allCourses.clear()
15     enrollments.allEnrollments.clear()
16     grades.bulkLoad()
17     courses.bulkLoad()
18     enrollments.bulkLoad()
19
20
21 def testListStudentsByCourseID_OneGrade():
22     loadTestData()
23     gradesResult = gradingHelper.listGradesByStudentID(6)
24     assert len(gradesResult) == 1
25     assert int(gradesResult[0][3]) == 90
26
27 def testListStudentsByCourseID_ManyGrades():
28     loadTestData()
29     gradesResult = gradingHelper.listGradesByStudentID(1)
30     assert len(gradesResult) == 8
31
32
```

```
StudentGradesApp > Python > GradingHelper.py > GradingHelper > listGradesBy
1 """File contains just one class, used to process grades"""
2
3 from Grades import Grades
4 from Courses import Courses
5 from Enrollments import Enrollments
6
7
8 class GradingHelper:
9     """Helper class used to combine data from grades, courses and enr
10
11     grades = Grades
12     courses = Courses
13     enrollments = Enrollments
14
15
16     def __init__(self, grades, courses, enrollments):
17         """Constructor"""
18         self.grades = grades
19         self.courses = courses
20         self.enrollments = enrollments
21
22
23     def listGradesByStudentID(self, student_id):
24         """Method to provide a list of grades for a specific student
25         students_grades = []
26         for current_enrollment in self.enrollments.allEnrollments:
27             current_student_id = int(current_enrollment.student_id)
28             if current_student_id == student_id:
29                 for current_course in self.courses.allCourses:
30                     if current_enrollment.course_id == current_course:
31                         for current_grade in self.grades.allGrades:
```

A more complex scenario also delivers significant benefits.

Review this code from a helper class, which uses several data objects. Writing code in this way allows the logic to be worked out and thoroughly tested in isolation.

Adding the new code to the broader application is done by mirroring the test code that was used to invoke it, making the development process much less taxing and encouraging refactoring as needed.

Sample Python Application (8 of 11)

GradingHelper.py •

StudentGradesApp > Python > GradingHelper.py > GradingHelper > listGradesByStudentID

```
1  from Grades import Grades
2  from Courses import Courses
3  from Enrollments import Enrollments
4
5  class GradingHelper:
6
7      ... grades = Grades
8      ... courses = Courses
9      ... enrollments = Enrollments
10
11     ... def __init__(self, grades, courses, enrollments):
12     ...         self.grades = grades
13     ...         self.courses = courses
14     ...         self.enrollments = enrollments
15     ...
16     ... def listGradesByStudentID(self, student_id):
17     ...         studentsGrades = []
18     ...         for currentEnrollment in self.enrollments.allEnrollments:
19     ...             currentStudentID = int(currentEnrollment.student_id)
20     ...             if currentStudentID == student_id:
21     ...                 for currentCourse in self.courses.allCourses:
22     ...                     if currentEnrollment.course_id == currentCourse.course_id:
23     ...                         for currentGrade in self.grades.allGrades:
24     ...                             if currentEnrollment.enrollment_id == currentGrade.enrollment_id:
25     ...                                 studentGrade = [currentEnrollment.student_id, currentCourse.course_name, \
26     ...                                                 currentGrade.assignment_id, currentGrade.mark]
27     ...                                 studentsGrades.append(studentGrade)
28     ...         return studentsGrades
```

Linting can really be classified as a code quality check.

The code on the right looks OK, but it is a little difficult to read and it scores poorly on a pylint check.

Sample Python Application (9 of 11)

```
shane@Shanes-MacBook-Pro Python % pylint GradingHelper.py
***** Module GradingHelper
GradingHelper.py:15:0: C0303: Trailing whitespace (trailing-whitespace)
GradingHelper.py:25:0: C0301: Line too long (151/100) (line-too-long)
GradingHelper.py:27:0: C0304: Final newline missing (missing-final-newline)
GradingHelper.py:1:0: C0114: Missing module docstring (missing-module-docstring)
GradingHelper.py:1:0: C0103: Module name "GradingHelper" doesn't conform to snake_case naming style (invalid-name)
GradingHelper.py:5:0: C0115: Missing class docstring (missing-class-docstring)
GradingHelper.py:16:4: C0116: Missing function or method docstring (missing-function-docstring)
GradingHelper.py:16:4: C0103: Method name "listGradesByStudentID" doesn't conform to snake_case naming style (invalid-name)
GradingHelper.py:17:8: C0103: Variable name "studentsGrades" doesn't conform to snake_case naming style (invalid-name)
GradingHelper.py:18:12: C0103: Variable name "currentEnrollment" doesn't conform to snake_case naming style (invalid-name)
GradingHelper.py:19:12: C0103: Variable name "currentStudentID" doesn't conform to snake_case naming style (invalid-name)
GradingHelper.py:21:20: C0103: Variable name "currentCourse" doesn't conform to snake_case naming style (invalid-name)
GradingHelper.py:23:28: C0103: Variable name "currentGrade" doesn't conform to snake_case naming style (invalid-name)
GradingHelper.py:25:32: C0103: Variable name "studentGrade" doesn't conform to snake_case naming style (invalid-name)
GradingHelper.py:18:8: R1702: Too many nested blocks (6/5) (too-many-nested-blocks)
GradingHelper.py:5:0: R0903: Too few public methods (1/2) (too-few-public-methods)

-----
Your code has been rated at 3.04/10 (previous run: 3.04/10, +0.00)
```

Many of the issues are related to readability, however readability matters so it is worthwhile to fix the highlighted issues.

Code is read 10 times more than it is written. *

* <https://www.goodreads.com/quotes/835238-indeed-the-ratio-of-time-spent-reading-versus-writing-is>

Sample Python Application (10 of 11)

```
GradingHelper.py M X
StudentGradesApp > Python > GradingHelper.py > GradingHelper > listGradesByStudentID
1  """File contains just one class, used to process grades"""
2
3  from Grades import Grades
4  from Courses import Courses
5  from Enrollments import Enrollments
6
7
8  class GradingHelper:
9  """Helper class used to combine data from grades, courses and enrollments data objects."""
10
11  """grades = Grades
12  """courses = Courses
13  """enrollments = Enrollments
14
15
16  def __init__(self, grades, courses, enrollments):
17  """ Constructor """
18  self.grades = grades
19  self.courses = courses
20  self.enrollments = enrollments
21
22  def listGradesByStudentID(self, student_id):
23  """ Method to provide a list of grades for a specific student id """
24  students_grades = []
25  for current_enrollment in self.enrollments.allEnrollments:
26  current_student_id = int(current_enrollment.student_id)
27  if current_student_id == student_id:
28  for current_course in self.courses.allCourses:
29  if current_enrollment.course_id == current_course.course_id:
30  for current_grade in self.grades.allGrades:
31  if current_enrollment.enrollment_id == current_grade.enrollment_id:
32  current_course_name = current_course.course_name
33  current_assignment_id = current_grade.assignment_id
34  current_mark = current_grade.mark
35  student_grade = [current_student_id, current_course_name, \
36  current_assignment_id, current_mark]
37  students_grades.append(student_grade)
38
39  return students_grades
```

With some edits, the readability improves significantly, as does the PyLint score.

Used across all files, the consistency in readability adds real value for developers.

A score of 8 is a common quality threshold, where anything below causes rejection of changes based on agreed team standards.

```
shane@Shanes-MacBook-Pro Python % pylint GradingHelper.py
```

```
***** Module GradingHelper
```

```
GradingHelper.py:1:0: C0103: Module name "GradingHelper" doesn't conform to snake_c
```

```
GradingHelper.py:22:4: C0103: Method name "listGradesByStudentID" doesn't conform to
```

```
GradingHelper.py:25:8: R1702: Too many nested blocks (6/5) (too-many-nested-blocks)
```

```
GradingHelper.py:8:0: R0903: Too few public methods (1/2) (too-few-public-methods)
```

```
-----
Your code has been rated at 8.46/10 (previous run: 8.08/10, +0.38)
```

Sample Python Application (11 of 11)

In addition to the Python code, we also need a database. Borrowing from earlier Maria DB implementations through Ansible, we build on that approach to create and populate a data source for this application.

Data is based on the unit test data, creating copies of csv files and using string concatenation in Excel to create insert statements for database tables.

The application will run as the user Bob on another server. By default, MySQL does not allow connections from remote servers, so this is addressed as part of the SQL script run to create and insert into tables.

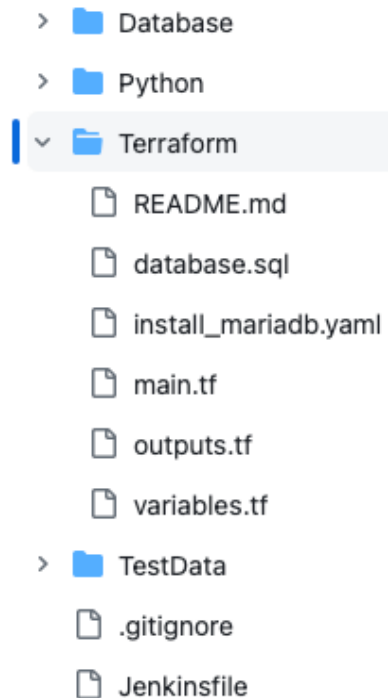
```
StudentGradesApp > Terraform > database.sql
1  -- Allow remote connections to bobdata by Bob
2  GRANT ALL PRIVILEGES ON bobdata.* to 'Bob'@'%' IDENTIFIED BY 'pa55word' with grant option;
3
4  Use bobdata;
5
6  --
7  -- Create & populate STUDENTS table
8  --
9  CREATE TABLE Students (
10     student_id INT PRIMARY KEY,
11     first_name VARCHAR(255),
12     surname VARCHAR(255),
13     date_of_birth DATE
14 );
15
16 INSERT INTO Students (student_id, first_name, surname, date_of_birth) VALUES (1, 'Ethan', 'S
17 INSERT INTO Students (student_id, first_name, surname, date_of_birth) VALUES (2, 'Olivia', '
18 INSERT INTO Students (student_id, first_name, surname, date_of_birth) VALUES (3, 'Liam', 'Pa
19 INSERT INTO Students (student_id, first_name, surname, date_of_birth) VALUES (4, 'Ava', 'Won
20 INSERT INTO Students (student_id, first_name, surname, date_of_birth) VALUES (5, 'Noah', 'Jo
21 INSERT INTO Students (student_id, first_name, surname, date_of_birth) VALUES (6, 'Emma', 'Ch
22 INSERT INTO Students (student_id, first_name, surname, date_of_birth) VALUES (7, 'Lucas', 'S
23 INSERT INTO Students (student_id, first_name, surname, date_of_birth) VALUES (8, 'Charlotte'
```

Jenkins Continuous Integration Pipeline for Sample Python API

High level overview of Exercise

There is a lot to cover in this exercise.

1. We will use the Python node configured earlier in the class to run a Continuous Integration pipeline for the Python application we reviewed in the last section.
2. We also have configured access to GitHub using a token as part of the Spring Boot sample, we will reuse that connection here.
3. The Jenkins pipeline is stored as code in GitHub with the application code and supporting artifacts.
4. The Terraform folder holds IaC to create a database and load with data created using insert statements dynamically created using spreadsheets in the Database folder.
5. The TestData folder is referenced when unit tests are run as



Exercise: CI Pipeline for Python API

With Jenkins up and running, the Python node connected and configured, and a GitHub account connected, we can progress to creating the pipeline job.

1. Create a new job, of type Pipeline, with name 'Python Grades API'.
2. Scroll down to Pipeline Definition, select your cloned repo URL and the credential you have created for access. If any errors are shown at this point, then review the setup work you have done with GitHub connection.

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/SMannionYorkU/CloudOps_Module4_Class8_Repo3_PythonGradesApp.git

Credentials ?

ShaneMannion/*****

Add

Advanced

Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/master

Exercise: CI Pipeline for Python API

3. Reviewing the pipeline file, it is quite simple.
 - a) Firstly, it installs the dependencies for the application that have been captured in requirements.txt
 - b) Next, pylint is run on all the python files in the Python folder. Pylint returns non-zero values as warning level notification, which is picked up by Jenkins as a failure. For this reason, the `--exit-zero` switch is applied.
 - c) Finally, unit tests are run as a separate stage.

```
1 pipeline {
2     agent {
3         node {
4             label 'Python-Node'
5         }
6     }
7     stages {
8         stage('Lint') {
9             steps {
10                 dir('Python') {
11                     sh 'pip3 install -r requirements.txt'
12                     sh 'pylint *.py --exit-zero'
13                 }
14             }
15         }
16         stage('Test') {
17             steps {
18                 dir('Python') {
19                     sh 'pytest'
20                 }
21             }
22         }
23     }
24     post {
25         success {
26             echo 'Job completed successfully'
27         }
28         failure {
29             echo 'Job failed'
30         }
31     }
32 }
```


Exercise: CI Pipeline for Python API

3. Reviewing the pipeline output, it is quite verbose as all the pip3 install and pylint information is captured.
 - a) Firstly, pip3 install adds all required packages, including pylint and pytest which are used as part of this pipeline.
 - b) The output shows a pylint score of 8.82, which is very good. Output from this command could be captured and used to fail the job the job if the score is below a threshold.
 - c) Finally, unit test output shows the number of tests run and time to execute (0.06s for 15 tests).

Your code has been rated at 8.82/10

```
[Pipeline] }
[Pipeline] // dir
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] dir
Running in /home/python-node/Jenkins/workspace/Python Grades API/Python
[Pipeline] {
[Pipeline] sh
+ pytest
===== test session starts =====
platform linux -- Python 3.9.16, pytest-7.1.0, pluggy-1.0.0
rootdir: /home/python-node/Jenkins/workspace/Python Grades API/Python
collected 15 items

test_Course.py . [ 6%]
test_Courses.py .. [ 20%]
test_Enrollment.py . [ 26%]
test_EnrollmentHelper.py . [ 33%]
test_Enrollments.py .. [ 46%]
test_Grade.py . [ 53%]
test_Grades.py .. [ 66%]
test_GradingHelper.py .. [ 80%]
test_Student.py . [ 86%]
test_Students.py .. [100%]

===== 15 passed in 0.06s =====
[Pipeline] }
[Pipeline] // dir
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Job completed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Exercise: CI Pipeline for Python API

3. Reviewing the pipeline output, it is quite verbose as all the pip3 install and pylint information is captured.
 - a) Firstly, pip3 install adds all required packages, including pylint and pytest which are used as part of this pipeline.
 - b) The output shows a pylint score of 8.82, which is very good. Output from this command could be captured and used to fail the job the job if the score is below a threshold.
 - c) Finally, unit test output shows the number of tests run and time to execute (0.06s for 15 tests).

Your code has been rated at 8.82/10

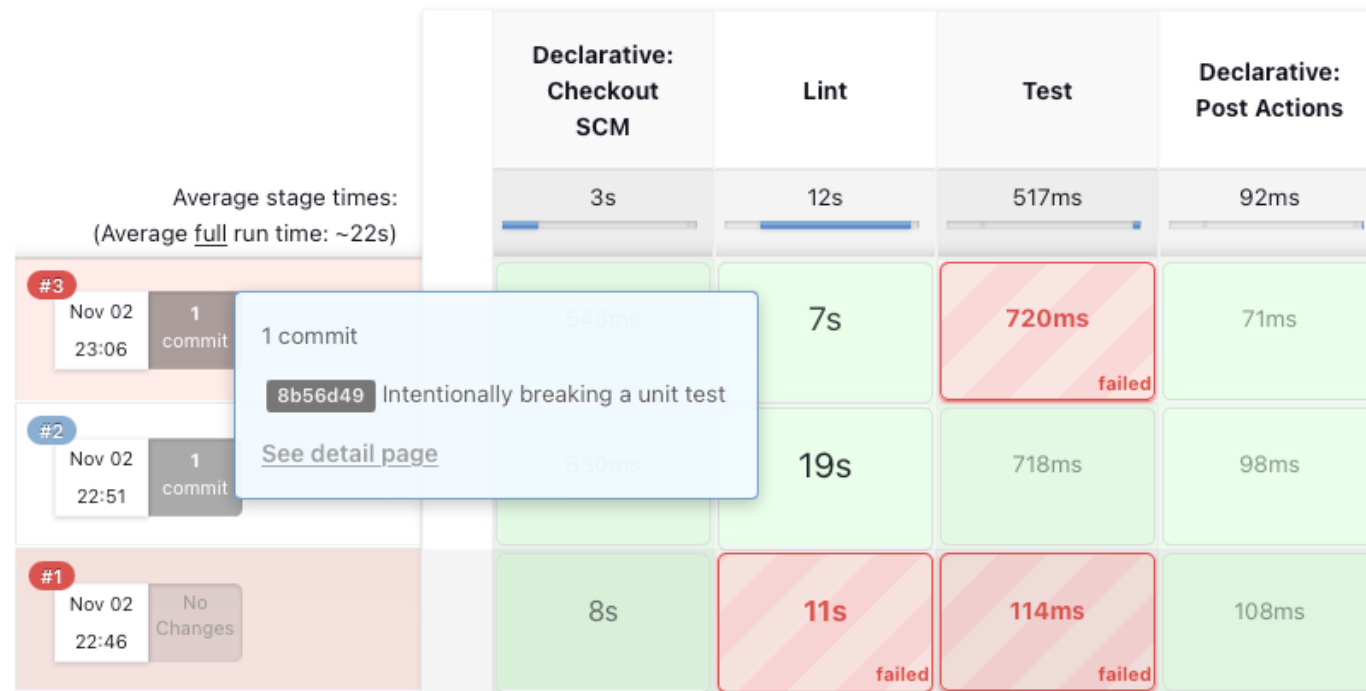
```
[Pipeline] }
[Pipeline] // dir
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] dir
Running in /home/python-node/Jenkins/workspace/Python Grades API/Python
[Pipeline] {
[Pipeline] sh
+ pytest
===== test session starts =====
platform linux -- Python 3.9.16, pytest-7.1.0, pluggy-1.0.0
rootdir: /home/python-node/Jenkins/workspace/Python Grades API/Python
collected 15 items

test_Course.py . [ 6%]
test_Courses.py .. [ 20%]
test_Enrollment.py . [ 26%]
test_EnrollmentHelper.py . [ 33%]
test_Enrollments.py .. [ 46%]
test_Grade.py . [ 53%]
test_Grades.py .. [ 66%]
test_GradingHelper.py .. [ 80%]
test_Student.py . [ 86%]
test_Students.py .. [100%]

===== 15 passed in 0.06s =====
[Pipeline] }
[Pipeline] // dir
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Job completed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

- Updating one of the unit tests to intentionally cause a test to fail subsequently causes the CI job to fail, as you can see from this screenshot of the console and the summary status below.

Stage View



```
+ pytest
===== test session starts =====
platform linux -- Python 3.9.16, pytest-7.1.0, pluggy-1.0.0
rootdir: /home/python-node/Jenkins/workspace/Python Grades API/Python
collected 15 items

test_Course.py . [ 6%]
test_Courses.py .. [ 20%]
test_Enrollment.py . [ 26%]
test_EnrollmentHelper.py . [ 33%]
test_Enrollments.py F. [ 46%]
test_Grade.py . [ 53%]
test_Grades.py .. [ 66%]
test_GradingHelper.py .. [ 80%]
test_Student.py . [ 86%]
test_Students.py .. [100%]

===== FAILURES =====
----- testBulkLoadEnrollments -----

def testBulkLoadEnrollments():
    enrollments.allEnrollments.clear()
> assert enrollments.bulkLoad() == 11
E assert 10 == 11
E + where 10 = <bound method Enrollments.bulkLoad of <Enrollments.Enrollments object at 0x7f36bfde3fa0>>()
E + where <bound method Enrollments.bulkLoad of <Enrollments.Enrollments object at 0x7f36bfde3fa0>> = <Enrollments.Enrollments object at 0x7f36bfde3fa0>.bulkLoad

test_Enrollments.py:11: AssertionError
----- Captured stdout call -----
RowCounter !> 1
===== short test summary info =====
FAILED test_Enrollments.py::testBulkLoadEnrollments - assert 10 == 11
===== 1 failed, 14 passed in 0.07s =====

[Pipeline] }
[Pipeline] // dir
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Job failed
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: script returned exit code 1
Finished: FAILURE
```

Journey Map

